```
//Program on Overcoupling

package com.red.down_coding;


class Animal
{
    public void sleep()
    {
        IO.println("Generic Animal is sleeping");
    }
}
class Dog extends Animal
{
    @Override
    public void sleep()
    {
        IO.println("Dog Animal is sleeping");
    }
}

class Lion extends Animal
{
    @Override
    public void sleep()
    {
        IO.println("Lion Animal is sleeping");
    }
}

public class LooseCoupling
{
    public static void main(String[] args)
    {
        Animal Animal = new Dog();
        checkAnimalBehavior(Animal); //Passing dog object
        IO.println("..........");
        Animal = new Lion();
        checkAnimalBehavior(Animal); //Passing lion object
    }

    public static void checkAnimalBehavior(Animal animal)
    {
        animal.sleep();
    }
}
```

```
//Program on Overcoupling :
package com.red.down.coding

class Animal
{
    public void sleep()
    {
        IO.println("Generic Animal is sleeping");
    }
}
class Dog extends Animal
{
    @Override
    public void sleep()
    {
        IO.println("Dog Animal is sleeping");
    }

    public void bark()
    {
        IO.println("Dog is barking");
    }
}

public class OvercoupLingExpander
{
    public static void main(String[] args)
    {
        Animal animal = new Dog();
        checkAnimalBehavior(Animal); //Passing dog object

        public static void checkAnimalBehavior(Animal animal)
        {
            // animal.bark(); // It is a compilation error
            // Line 1 is valid but Dog.bark() method does not exist in Animal class
            // Line 2 is valid but Dog.bark() method is inherited to Animal
            // if the ref is Dog-Dog.bark() method
            {
                Dog dog = (Dog) animal; //Down-casting
                dog.sleep();
                dog.bark();
            }
        }
}
```

```
package com.red.down_coding

class Animal
{
    public void sleep()
    {
        IO.println("Generic Animal is sleeping");
    }
}
class Dog extends Animal
{
    @Override
    public void sleep()
    {
        IO.println("Dog Animal is sleeping");
    }

    public void bark()
    {
        IO.println("Dog is barking");
    }
}
class Lion extends Animal
{
    @Override
    public void sleep()
    {
        IO.println("Lion Animal is sleeping");
    }

    public void roar()
    {
        IO.println("Lion is roaring");
    }
}

public class OvercoupLingExpander
{
    public static void main(String[] args)
    {
        Animal animal = new Dog();
        checkAnimalBehavior(animal); //Passing dog object

        IO.println("..........");

        animal = new Lion();
        checkAnimalBehavior(animal);

    public static void checkAnimalBehavior(Animal animal)
    {
        if(animal instanceof Dog)
        {
            Dog dog = (Dog) animal;  //UCD approach
            dog.sleep();
            dog.bark();
        }
        else if(animal instanceof Lion lion) //JDK 14+
        {
            lion.sleep();
            lion.roar();
        }
    }
}
```

```
package com.red.down_coding.

class Payment
{
    public void makePayment()
    {
        IO.println("Making a generic Payment");
    }
}

class UPI extends Payment
{
    public void makePayment()
    {
        IO.println("Making a Payment through UPI");
    }

    public void offer()
    {
        IO.println("Take a 100 payment and get 2000 RS cash back");
    }
}

class DebitCard extends Payment
{
    public void makePayment()
    {
        IO.println("Making a Payment through Debit Card");
    }
}

class CreditCard extends Payment

    public void makePayment()
    {
        IO.println("Making a Payment through Credit Card");
    }

    public void offer()
    {
        IO.println("Making a Payment of 2 lakh and get 2 days holidays in ODC");
    }
}

public class PaymentTypeMent {
    public static void main(String[] args) throws InterruptedException,
ClassNotFoundException, ClassCastException
    {
        //Passing class name (Q8) through command line argument
        Payment payment = null;
        payment = (Payment) Class.forName(args[0].newInstance();
        payMentMethod(payment);

        Payment payment = null;
        payment = new UPI();
        payMentMethod(payment);
        IO.println("..........");
        payment = new CreditCard();
        payMentMethod(payment);
        IO.println("..........");
        payment = new DebitCard();
        payMentMethod(payment);

    }

    public static void payMentMethod(Payment payment)
    {
        if(payment instanceof UPI upi)
        {
            upi.makePayment();
            upi.offer();}

        else if(payment instanceof CreditCard card)
        {
            card.makePayment();
            card.offer();}
        else if(payment instanceof DebitCard card)
        {
            card.makePayment();}
    }
}
```

instanceof Operator :

* It is a relational operator so, will return true OR false. It is also a keyword.
* It is used to verify **whether a reference variable is pointing** to a particular type of object or not ?
* It between **reference variable** and **class OR interface type.** We must have IS-A relation otherwise we will get a compilation error.
* It is used to create the **Inner ClassCastException**.

//Program

```
package com.sun.instanceof_opr;

class Alpha
{
}

class Beta extends Alpha
{
}

class Gamma extends Beta
{
}

public class InstanceOfDemo
{
    public static void main(String[] args)
    {
        Gamma g = new Gamma();
        if(g instanceof Gamma)
            sp.println("g is pointing to gamma object");
        if(g instanceof Beta)
            sp.println("g is pointing to beta object");
        if(g instanceof Alpha)
            sp.println("g is pointing to alpha object");
        if(g instanceof Object)
            sp.println("g is pointing to object class object");
    }
}


package com.red_Infinite3_opr;
public class InstanceOfDemo2
{
    public static void main(String[] args)
    {
        Integer i = 10;
        if(i instanceof Integer)
        {
        IO.println("..........");
        }
        String s = "Hello";
        if(s instanceof Array)
            sp.println("str is pointing to string object");
    }
}

package com.red.instanceof_opr;
class Acc
{
    public void f1()
    {
        IO.println("Acc car f1");
    }
}

class Ford
{
    public void m1()
    {
        IO.println("Ford car m1");
    }
}

public class InstanceOfDemo3
{
    public static void main(String[] args)
    {
        Acc[] car = new Acc[4];
        if(car[0] instanceof Ford) //error

        {

        }
    }
}
```

```
package com.red.instanceof_opr;
class Vehicle
{
    public void run()
    {
        System.out.println("Vehicle is running");
    }
}

class BMW extends Vehicle
{
}

class Audi extends Vehicle
{
}


public class InstanceOfDemo4
{

    public static void main(String[] args)
    {
        //BMW b = new BMW();
        //Vehicle v1[0] = b;

        Audi a = new Audi();
        a.run();//error
    }

    public static void acceptCarOperand(Vehicle v)
    {

        if( v instanceof BMW)
        {
            System.out.println("BMW car");
        }
        else
        {
            System.out.println("Audi car");
        }
    }
}
```

```
**Q:**
package com.edi.dis;
class Person
{
    private int age = 18;
    public int getage();
    {
        return this.age;
    }
    public void printage();
    {
        IO.println(this.getage());
    }
}
class Student extends Person
{
    private int age = 21;
    public int getage();
    {
        return this.age;
    }

    public void printage();
    {
        IO.println(this.getage());
    }
}

public class Q
{
    public static void main(String[] args)
    {
        Person s = new Student();
        s.printage(); //18
    }
}

package com.edi.dis;
class Person
{
    private int age = 21;
    public int getage();
    {
        return this.age;
    }

    public void printage();
    {
        IO.println(this.getage());
    }
}

class Student extends Person
{
    private int age = 21;

    public int getage();
    {
        return this.age;
    }

    public void printage();
    {
        IO.println(super.getage());
    }

}

public class Q
{
    public static void main(String[] args)
    {
        Person s = new Student();
        s.printage(); //18
    }
}
```